

Universe Reinsurance Platform — Repository Migration & Database Alignment Guide

Switching the deployment source from `Darchville-Analytics/modelling_tool` to `iwadaya/Saudi_re`, seeding the database, and proving the application is linked to — and aligned with — its database.

Version 1.0 — September 2026 — Confidential, for internal IT use only.
Companion to the *Work Environment Deployment Specification v2.0* (July 2026) and the *Redeployment Quick Reference* (August 2026). Where this guide and the v2.0 specification differ, this guide is current.
Source repository: `https://github.com/iwadaya/Saudi_re` (branch `main`). Living companions in the repository: `DEPLOYMENT.md`, `deploy/README.md`, `docs/seed-test-treaties.md`.

1. What this guide covers, and what changed	6. Path B — fresh server from the new repository (deployment kit)
2. Before you start	7. Checklist — is the database linked to the app, and does it align?
3. GitHub access to the new repository	8. Troubleshooting the switch
4. Path A — switch the existing server to the new repository	Appendix A — Quick command reference (v2.0 layout, PM2)
5. Seeding the database	Appendix B — Sign-off record

1. What this guide covers, and what changed

The server was stood up with the v2.0 specification: Ubuntu, the application checked out at `/opt/universe` as the `universe` user, one `.env` file at `/opt/universe/.env`, PM2 running `ecosystem.config.cjs`, nginx terminating TLS in front of port 4000, and PostgreSQL holding the `universe` database. **None of that changes.** What changes is *where the code comes from*: the deployment source moves from the organisation repository `Darchville-Analytics/modelling_tool` to `iwadaya/Saudi_re`.

The new repository contains the full history of the old one plus a maintained deployment kit, so switching is a fast-forward of the same code line, not a migration to a different product. Two things in the new repository behave differently from what the v2.0 specification describes, and both matter on the day you switch:

Area	v2.0 specification (modelling_tool)	Now (iwadaya/Saudi_re)
Repository and access	Org-owned private repo. Fine-grained PAT with resource owner <i>Darchville-Analytics</i> , or a deploy key on <code>modelling_tool</code> .	Private repo under the personal account iwadaya . The old token or deploy key does not open it — you need a new fine-grained PAT (resource owner <i>iwadaya</i> , repository <i>Saudi_re</i> , Contents: read) or a new deploy key added to <code>Saudi_re</code> (§3).
deploy.sh	Pulled <code>main</code> , ran <code>npm ci</code> , built, migrated, reloaded PM2. Run as the app user.	The root <code>deploy.sh</code> is now a one-line wrapper that runs deploy/deploy.sh : <code>fetch/checkout</code> → <code>npm ci</code> → <code>build</code> → <code>migrate:status</code> + <code>migrate:up</code> → reference-data seed → <code>systemctl restart universe</code> → smoke test. It is written for sudo (it drops to <code>universe</code> for the Git/npm/build/migrate/seed steps itself), and it reads <code>/etc/universe/universe.env</code> by default. On a PM2 server you therefore run it with <code>ENV_FILE=/opt/universe/.env</code> and <code>--no-restart</code> , then reload PM2 yourself (§4, step 5).

Area	v2.0 specification (modelling_tool)	Now (iwadaya/Saudi_re)
Reference-data seeding	Happened on application boot only (ensureReferenceData), plus the optional SEED_ON_DEPLOY=reference hook.	Still happens on boot, and is an explicit deploy step: node server/src/db/seeds/run.js (npm run seed:reference). It adds the GCC market extras (Saudi/GCC cedants and reinsurers) and refuses to touch contracts, quotes, claims or users. npm run seed:reference:check reports the counts without writing anything (§5).
Migrations	Same runner and commands.	Same chain (_migrations table keyed by filename). This snapshot ships 149 migration files, the last being 157_quote_offer_status_checks.sql . Nothing is re-applied; only files your database has not seen run.
Deployment kit	—	New deploy/ directory: install-server.sh , provision-db.sh , deploy.sh , smoke-test.sh , a hardened systemd unit, an nginx site and an environment template — the maintained path for a fresh server (§6).
Smoke test	Manual curl .	deploy/smoke-test.sh : shallow health, deep health (DB round-trip), user list served from the database, SPA served, optional real login. Exit 0 = green.

Everything else in the v2.0 specification — sizing, ports, nginx, TLS, backups, secrets, PM2, the security baseline — still applies as written.

Two paths. §4 switches the **existing** server in place (recommended: no re-provisioning, no data movement, ~15 minutes). §6 builds a **fresh** server from the new repository with the deployment kit and moves the data across. Both end at the same checklist in §7.

2. Before you start

Collect these facts first; several steps depend on them.

Item	How to find it	Write it here
Runtime in use	pm2 status (as the universe user) shows an online universe process → PM2 (v2.0). systemctl status universe shows active (running) → systemd (deployment kit).	
Application root and env file	v2.0: /opt/universe and /opt/universe/.env . Kit: /opt/universe and /etc/universe/universe.env (symlinked to /opt/universe/.env).	
Currently deployed commit	git -C /opt/universe log -1 --format='%h %ci %s'	
Current remote	git -C /opt/universe remote -v — expect Darchville-Analytics/modelling_tool	
Migration state before the switch	sudo -u universe -H npm run migrate:status --prefix server from /opt/universe → the last lines must read Pending (0): . If anything is pending <i>before</i> you start, resolve that first.	
Node.js seen by root and by universe	node -v and sudo -u universe -H node -v → both Node 20 or newer, ideally the same version. deploy/deploy.sh builds and migrates with root's node ; PM2 runs the app with the universe user's.	

Item	How to find it	Write it here
pm2 on the universe user's PATH	<code>sudo -u universe -H pm2 -v</code> → prints a version (v2.0 installed PM2 globally). If it prints <code>command not found</code> , use <code>/opt/universe/node_modules/.bin/pm2</code> wherever this guide says <code>pm2</code> .	
Database	<code>DATABASE_URL</code> in the env file (host, port, database name, role). <code>psql "\$DATABASE_URL" -c 'select 1'</code> must work from the app server.	
Who owns GitHub access	A GitHub account with read access to <code>iwadaya/Saudi_re</code> to mint the token or approve the deploy key (§3).	
Seeding decision	Which reference layers you want on this server (§5.1). Default: base catalogue + GCC extras (automatic), no Ghana pack, never the fabricated test portfolio.	
Maintenance window	The PM2 reload is zero-downtime, but migrations may hold locks for a few seconds. Choose a quiet moment.	

Also confirm outbound HTTPS from the server to `github.com` (or SSH port 22 if you use a deploy key): `curl -sI https://github.com | head -1` should print `HTTP/2 200`.

3. GitHub access to the new repository

GitHub does not accept account passwords for Git operations, and credentials are scoped to a repository — the token or key that opened `Darchville-Analytics/modelling_tool` will fail against `iwadaya/Saudi_re` with `remote: Repository not found` or `Permission denied`. Set up one of the two options below **before** touching the checkout.

3.1 Option 1 — fine-grained personal access token (HTTPS, recommended)

1. On GitHub, signed in as `iwadaya` — the account that owns the repository. A fine-grained token can only reach repositories owned by the account (or organisation) that creates it, so a collaborator cannot mint one for `iwadaya/Saudi_re`; a collaborator uses a deploy key (§3.2) or a classic token with the `repo` scope instead. Go to **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens** → **Generate new token**.
2. **Resource owner:** `iwadaya`. **Repository access:** *Only select repositories* → `Saudi_re`. **Permissions** → **Repository permissions** → **Contents: Read-only** (Metadata: Read is added automatically). Expiry: 90 days; put the rotation date in your calendar (v2.0 §9.2).
3. On the server, remove the stored token for the old repository so Git prompts for the new one, then test access **interactively once** as the deploy user so the new token is stored and later deploys never prompt:

```
# Remove the old github.com line from the deploy user's credential store (if any).
sudo -u universe -H bash -c 'test -f ~/.git-credentials && sed -i "/github\.com/d" ~/.git-credentials; git config --global credential.helper store'

# Test read access to the NEW repository. Username = your GitHub username, password = the new token.
sudo -u universe -H git ls-remote --heads https://github.com/iwadaya/Saudi_re.git main
# <40-hex commit id> refs/heads/main ← success; the token is now saved in
~universe/.git-credentials (mode 600)
```

3.2 Option 2 — SSH deploy key

A GitHub deploy key can be attached to **one** repository only, so the key on `modelling_tool` cannot simply be reused. Generate a second key and give it its own host alias:

```
sudo -u universe -H ssh-keygen -t ed25519 -C "universe-deploy-saudi_re" -N "" -f
/home/universe/.ssh/id_ed25519_saudi_re
sudo -u universe -H cat /home/universe/.ssh/id_ed25519_saudi_re.pub
# → GitHub: iwadaya/Saudi_re → Settings → Deploy keys → Add deploy key (leave "Allow write
access" unticked)

sudo -u universe -H tee -a /home/universe/.ssh/config >/dev/null <<'EOF'
Host github.com-saudi_re
  HostName github.com
  IdentityFile /home/universe/.ssh/id_ed25519_saudi_re
  IdentitiesOnly yes
EOF
sudo -u universe -H chmod 600 /home/universe/.ssh/config
sudo -u universe -H ssh -T git@github.com-saudi_re # "Hi iwadaya/Saudi_re! You've
successfully authenticated..."
```

With this option the remote URL in §4 step 3 is `git@github.com-saudi_re:iwadaya/Saudi_re.git` instead of the HTTPS URL.

4. Path A — switch the existing server to the new repository

All commands assume the v2.0 layout (`/opt/universe` , user `universe` , PM2). Run them from an admin shell with `sudo` ; where a command must run as the application user it is prefixed with `sudo -u universe -H` .

Step 1 — Take a backup and record the starting point

```
cd /opt/universe
git log -1 --format='%h %ci %s' # currently deployed commit – keep this
for rollback
sudo -u universe -H npm run migrate:status --prefix server | tail -n 3 # must end with
"Pending (0):"

# Full database dump with the checked-in script (same as the nightly job). Prints the dump
path.
sudo install -d -o universe -g universe -m 750 /var/backups/universe # harmless if it
already exists; universe cannot create it itself
sudo -u universe -H bash -c 'cd /opt/universe && set -a && . ./env && set +a &&
BACKUP_DIR=/var/backups/universe scripts/backup-db.sh'
ls -lt /var/backups/universe | head -3 # the new .dump file is at the top
```

If `BACKUP_DIR` differs on your server, use the value from your cron entry (v2.0 §4.3). Do not continue without a dump you can see on disk.

Step 2 — Confirm access to the new repository

Complete §3 and make sure `git ls-remote` (or `ssh -T`) succeeded as the `universe` user. `deploy/deploy.sh` fetches as that user and cannot answer a password prompt.

Step 3 — Point the checkout at iwadaya/Saudi_re

```
# HTTPS (Option 1). For a deploy key (Option 2) use: git@github.com-
saudi_re:iwadaya/Saudi_re.git
sudo -u universe -H git -C /opt/universe remote set-url origin
https://github.com/iwadaya/Saudi_re.git
sudo -u universe -H git -C /opt/universe remote -v                # both lines now
show iwadaya/Saudi_re
sudo -u universe -H git -C /opt/universe fetch --prune --tags origin  # replaces the
old origin/* refs

# What you are about to deploy:
sudo -u universe -H git -C /opt/universe log --oneline HEAD..origin/main | wc -l      #
number of new commits
sudo -u universe -H git -C /opt/universe diff --stat HEAD origin/main --
server/src/db/migrations | tail -n 1      # new migration files, if any
sudo -u universe -H git -C /opt/universe merge-base --is-ancestor HEAD origin/main && echo
"OK: fast-forward" || echo "WARNING: deployed commit is not in the new history – read §7.4
(orphan check) before migrating"
```

Nothing has changed on disk yet apart from the remote name; the running application is untouched.

Step 4 — Check the environment file

The variables from v2.0 remain valid; the new repository needs nothing new. Confirm the critical ones (passwords are masked in the output):

```
sudo -u universe -H bash -c 'cd /opt/universe && ls -l .env && grep -E
"^(NODE_ENV|PORT|DATABASE_URL|CORS_ORIGIN|RUN_MIGRATIONS_ON_BOOT|SEED_ON_DEPLOY|ALLOW_LOCAL_U
PLODS|ALLOW_DEMO_AUTH)=" .env | sed -E "s#(://[^\:]+:)[^\@]*@#\1***@#"'
```

Expected: `-rw-----` owned by `universe`; `NODE_ENV=production`; `PORT=4000`; exactly one `DATABASE_URL`; `RUN_MIGRATIONS_ON_BOOT=false` (or absent — the default is false); `ALLOW_DEMO_AUTH` absent; `SEED_ON_DEPLOY` absent, or `reference` if you want the Ghana pack (§5.1). If you want to add `SEED_ON_DEPLOY=reference`, do it now — the deploy step reads it.

Step 5 — Deploy from the new repository

Option A1 — the repository's deploy script (recommended). It does every step in order, stops at the first failure, and runs the `Git/npm/build/migrate/seed` steps as the `universe` user. On a PM2 server pass the env-file location and skip the systemd restart:

```
sudo env ENV_FILE=/opt/universe/.env /opt/universe/deploy/deploy.sh --ref main --no-restart
```

What it prints, in order (each step is a ► header): 1. fetching origin and checking out 'main' (and the commit it landed on) → 2. installing dependencies from lockfiles (`npm ci`) → 3. building the client bundle → 4. database migrations (the applied/pending list, then one migration completed line per file and migrations complete) → 5. reference data (countries, currencies, brokers, cedants, ...) — never contracts (the counts block shown in §5.2, then the `seed-on-deploy:` line) → 6. restart skipped (`--no-restart`) — run: `sudo systemctl restart universe && deploy/smoke-test.sh` → done — `<commit>` is deployed. **Ignore the `systemctl` hint in step 6 on a PM2 server** — reload PM2 as shown next. Normal noise between the headers: a `dotenv` banner (`<diamond>` injected env (N) from `../.env`) before every Node step, `JSON DB pool connecting` / `DB pool configured` lines, and inside step 5 an early reference data ready ... `"cedants":22` line *before* the counts block reports 40 — the block is the

authoritative result. The migrations and the seed run **before** the reload, so a failure leaves the old version running.

Now reload the application under PM2 — as the user that owns the PM2 daemon (`universe` per v2.0), with the same `pm2` you started it with, and with the same environment overrides (`PM2_INSTANCES` , `DB_POOL_MAX`) exported if you used any at start, because `ecosystem.config.cjs` re-derives the worker count and per-worker pool on every reload — and smoke-test it:

```
sudo -u universe -H bash -c 'cd /opt/universe && pm2 reload ecosystem.config.cjs && pm2 save'
# zero-downtime rolling reload (= npm run cluster:reload)
sudo -u universe -H pm2 status
# every universe worker "online", uptime just reset
BASE_URL=http://127.0.0.1:4000 /opt/universe/deploy/smoke-test.sh
# must end with PASSED
```

Option A2 — the same steps by hand (when each step must be inspected individually):

```
sudo -u universe -H bash                                # open a shell as the application user
cd /opt/universe
git fetch --prune --tags origin
git checkout -B main origin/main
git log -1 --format='%h %ci %s'
npm ci --include=dev && npm ci --include=dev --prefix client && npm ci --prefix server # --
include=dev: the client build needs vite even with NODE_ENV=production
npm run build --prefix client
npm run migrate:status --prefix server                  # review the pending list before applying it
npm run migrate:up --prefix server                      # apply BEFORE reloading the app
node server/src/db/seeds/run.js                        # reference data – prints the counts block
($5.2); aborts if any business table changed
npm run seed:deploy --prefix server                    # Ghana pack if SEED_ON_DEPLOY=reference;
otherwise prints "skipping" (expected)
pm2 reload ecosystem.config.cjs && pm2 save             # zero-downtime PM2 reload (= npm run
cluster:reload)
exit
BASE_URL=http://127.0.0.1:4000 /opt/universe/deploy/smoke-test.sh
```

On a systemd (deployment-kit) server the whole step is simply `sudo /opt/universe/deploy/deploy.sh -ref main` — it reads `/etc/universe/universe.env` , restarts the service and runs the smoke test itself.

Step 6 — Verify

Run the checklist in §7. At minimum, before you tell users: `Pending (0)` from `migrate:status` , `PASSED` from the smoke test, `db.ok:true` from `/api/health/deep` **through nginx** (`curl -fsS https://<APP_DOMAIN>/api/health/deep`), the seed counts from `npm run seed:reference:check` , and one real login over HTTPS.

Step 7 — After the switch

- `pm2 save` was run (step 5), so the process list survives a reboot; `pm2 startup` from v2.0 is unchanged.
- The backup cron, nginx site and TLS certificate reference paths, not the repository — nothing to change. Run `scripts/backup-db.sh` once more now so the newest dump reflects the migrated schema.
- Delete or let expire the old token/deploy key for `modelling_tool` ; record the new token's expiry.
- Add a line to your deployment log: date, old commit → new commit, migrations applied (from step 5 output), seed counts, who signed off (Appendix B).

Rollback

The previous commit is still in the local repository and in the new repository's history:

```
sudo env ENV_FILE=/opt/universe/.env /opt/universe/deploy/deploy.sh --ref <old commit from  
step 1> --no-restart --no-seed  
sudo -u universe -H bash -c 'cd /opt/universe && pm2 reload ecosystem.config.cjs'
```

Migrations are forward-only. If a migration itself is the problem, stop the app (`sudo -u universe -H pm2 stop universe`), restore the step-1 dump (`sudo -u postgres pg_restore --clean --if-exists -d universe /var/backups/universe/<file>.dump`), then redeploy the old commit as above.

5. Seeding the database

5.1 What gets seeded, by whom, and when

"Seeding" is four separate layers. Only the first three are ever appropriate on this server.

Layer	What it writes	When it runs	Writes contracts?
Migrations (<code>npm run migrate:up</code>)	The schema, and the base catalogue that ships inside it: 75 countries with regions (GCC, Levant, North Africa, Sub-Saharan Africa incl. Ghana, Europe, Americas, South/Southeast/East Asia — not every country in the world; add others on the reference screens), 32 currencies each with a USD rate, inflation (CPI) series for Saudi Arabia, the UAE and the UK (2000–2026), the CRESTA-zone table (empty until zones are added), facultative reference tables and taxonomies, treaty types, classes of business, roles and the mandate hierarchy, and six generic demo personas (<code>chief.underwriter</code> , <code>retro.manager</code> , <code>underwriter1–4</code> , password <code>demo2026</code> on a fresh database).	Deploy step 4. Each file runs once, recorded in <code>public._migrations</code> .	No
Boot seed (<code>ensureReferencedata</code>)	The canonical lists: 45 countries, 32 currencies, 11 brokers, 11 treaty types, 14 classes of business, 15 reinsurers, 22 cedants. Every insert is <code>WHERE NOT EXISTS</code> .	Every application start, and inside the reference seed.	No
Reference seed (<code>node server/src/db/seeds/run.js</code> , <code>npm run seed:reference</code>)	The boot seed plus <code>002_reference_data.sql</code> : the canonical mirror and the GCC market extras — 18 cedants (Tawuniya, MEDGULF, Al Rajhi Takaful, Bupa Arabia, Walaa, SALAMA, Orient, Sukoon, GIG Bahrain, Solidarity, Kuwait Insurance, Gulf Insurance Group, Misr, GIG Egypt, Dhofar, National Life & General, ...) and 10 reinsurers. Idempotent (upserts on natural keys from migration 150).	Deploy step 5 in <code>deploy/deploy.sh</code> (skip with <code>--no-seed</code>), or by hand at any time.	No — it counts <code>contract</code> , <code>quote</code> , <code>claim</code> and <code>uw_user</code> before and after and exists non-zero if any of them changed .

Layer	What it writes	When it runs	Writes contracts?
Ghana pack (<code>npm run seed:deploy</code> with <code>SEED_ON_DEPLOY=reference</code>)	Ghana country metadata, the GHS currency and its USD rate, the Ghana CPI series 2000–2026, 8 Ghana CRESTA zones, 8 Ghanaian cedants. Idempotent.	Deploy step 5 (after the reference seed), only while <code>SEED_ON_DEPLOY=reference</code> is in the env file. Otherwise the hook prints <code>seed-on-deploy: SEED_ON_DEPLOY is not set – skipping.</code>	No
Test portfolio (<code>SEED_ON_DEPLOY=1 / reset</code> , or <code>npm run seed:treaties</code>)	100 fabricated treaties with full pricing, triangles, workflow and audit rows.	Only when someone sets those values.	YES — test/demo databases only. Never on this server.

Decision for this server. Leave `SEED_ON_DEPLOY` unset unless the Ghana lookups are wanted; if they are, add exactly one line `SEED_ON_DEPLOY=reference` to `/opt/universe/.env` (no restart needed — the deploy hook reads the file). Never set `1` or `reset` here. If fabricated treaties ever appear, remove only them with `node server/scripts/seedTestTreaties.js --reset-only` (deletes rows stamped `seed:test-treaties`, nothing else).

5.2 What "seeded correctly" looks like

Verified against a freshly migrated PostgreSQL 16 database at this commit. The reference seed prints this block; `npm run seed:reference:check` prints the same counts read-only under the headings `Reference data (read-only check):` and `Business data (never written by this script):`.

Reference data after seeding:

```
countries      75
currencies     32
brokers        11
reinsurers     25
cedants        40
treaty_types   11
classes_of_business 15
roles          9
```

Business data (unchanged):

```
contracts  0
quotes     0
claims     0
users      6
```

On your existing database expect **at least** these reference counts: reinsurers rise from 15 to 25 and cedants from 22 to 40 the first time the reference seed runs (the GCC extras), the Ghana pack adds 1 currency and 8 cedants, and anything users have created on the reference screens adds to the totals. The business rows (`contracts` , `quotes` , `claims` , `users`) must show **your real numbers, unchanged** by the seed — that is the invariant the script enforces.

Spot checks that prove the rows landed in *this* database and are visible to the app:


```
cd /opt/universe && set -a && . /.env && set +a
psql "$DATABASE_URL" -c "SELECT company_name FROM public.companies c JOIN public.country k
USING (country_id) WHERE k.country_code='SA' AND c.is_active IS NOT FALSE ORDER BY 1"
#   Tawuniya, MEDGULF, Al Rajhi Takaful, Bupa Arabia, Walaa Insurance, SALAMA, Malath
Insurance, Gulf Union Insurance ...
psql "$DATABASE_URL" -tAc "SELECT count(*) FROM public.contract WHERE import_metadata-
>>'source'='seed:test-treaties'"    # 0 – no fabricated treaties
psql "$DATABASE_URL" -tAc "SELECT count(*) FROM public.currency WHERE currency_code='GHS'"
# 1 only if the Ghana pack was requested
```

In the browser: open a new treaty — the **Cedant** dropdown lists the Saudi and GCC companies above, the **Currency** dropdown includes SAR/AED/KWD, and the **Reinsurer** panel includes Qatar Re, Trust Re and CCR Re.

5.3 Re-running the seed, and what can go wrong

- The seed is idempotent: run `sudo -u universe -H node server/src/db/seeds/run.js` from `/opt/universe` as often as you like.
- It must run **after** `migrate:up`. On a database that has not reached migration 150 it fails with `there is no unique or exclusion constraint matching the ON CONFLICT specification` — apply migrations, then re-run.
- If it aborts with `seed:reference changed business table "..."`, something other than reference data moved during the run (most likely a user working at that moment). Nothing is rolled back that users did; re-run at a quiet moment and compare the two counts.
- A seed failure inside `deploy/deploy.sh` stops the script before the reload — the old version keeps running. Fix, then re-run the deploy.

6. Path B — fresh server from the new repository (deployment kit)

Use this when you want a clean box rather than switching in place. The kit is documented step by step in `deploy/README.md` (systemd + nginx + PostgreSQL 16, reference data only). The outline, with the data move from the old server:

B1 — Prepare the host. Ubuntu 22.04/24.04, a sudo user, DNS for `APP_DOMAIN`, the TLS certificate at hand. The kit already points at the new repository; the clone prompts for your GitHub username and the fine-grained token (§3.1):

```
git clone https://github.com/iwadaya/Saudi_re.git /tmp/universe-kit
sudo APP_DOMAIN=universe.example.internal REPO_URL=https://github.com/iwadaya/Saudi_re.git
GIT_REF=main \
    bash /tmp/universe-kit/deploy/install-server.sh
```

It installs Node 20, PostgreSQL 16 and nginx, creates the `universe` user, clones the repository into `/opt/universe`, creates the database role and database, writes `/etc/universe/universe.env` with generated secrets and the ready `DATABASE_URL`, and installs the systemd unit and nginx site. Review the env file (`sudo -e /etc/universe/universe.env`), in particular `CORS_ORIGIN`.

B2 — Move the data (optional; skip for an empty pilot database). On the old server take a fresh dump (`scripts/backup-db.sh`), copy the `.dump` file and the uploads directory (`/opt/universe/uploads`, or your `UPLOAD_DIR`) across, then on the new server restore **before** the first deploy so the migrations bring the restored schema forward:

```
sudo systemctl stop universe 2>/dev/null || true
sudo -u postgres pg_restore --clean --if-exists --no-owner --role=universe -d universe
/path/to/universe-<stamp>.dump
sudo rsync -a --chown=universe:universe old-server:/opt/universe/uploads/
/var/lib/universe/uploads/
```

B3 — Build, migrate, seed, start, smoke-test in one command:

```
sudo /opt/universe/deploy/deploy.sh --ref main
```

The expected tail of the seed output is the block in §5.2 (with your real business counts if you restored data), followed by `systemctl restart universe` and the smoke test ending in `PASSED`.

B4 — Install the real TLS certificate (`deploy/README.md` step 7), then run the §7 checklist through nginx: `BASE_URL=https://<APP_DOMAIN> /opt/universe/deploy/smoke-test.sh`.

B5 — Cut over. Point DNS at the new server, make sure `CORS_ORIGIN` is exactly the URL users type, restart (`sudo systemctl restart universe`), install the backup cron (`deploy/README.md` step 10), and decommission the old server after the retention period.

The seeded demo personas (`demo2026`) exist on a fresh database: rotate or deactivate them with `server/scripts/manage-user.js` before sharing the URL (`deploy/README.md` step 9). A restored production database keeps your real users instead.

7. Checklist — is the database linked to the app, and does it align?

Work through the groups in order. "Command" runs on the application server; `psql` commands need `DATABASE_URL` in the shell — load it once with `cd /opt/universe && set -a && . ./env && set +a` (as `universe` or `root`). Tick every row before sign-off.

7.1 Configuration — the app is told about exactly one database

#	Check	Command	Expected	Proves
C1	One env file, locked down	<code>ls -l /opt/universe/.env</code>	<code>-rw----- universe universe</code>	Secrets readable by the app user only
C2	Exactly one <code>DATABASE_URL</code>	<code>grep -c '^DATABASE_URL=' /opt/universe/.env</code>	1	No duplicate/overridden connection string
C3	No placeholders	<code>grep -c CHANGE_ME /opt/universe/.env</code>	0 (grep exits 1 when the count is 0 — expected)	Real values everywhere
C4	Production mode, migrations explicit	<code>grep -e '^NODE_ENV=' -e '^RUN_MIGRATIONS_ON_BOOT=' /opt/universe/.env</code>	<code>NODE_ENV=production , RUN_MIGRATIONS_ON_BOOT=false</code> (or absent)	Schema changes only happen in the deploy step
C5	Seeding posture	<code>grep '^SEED_ON_DEPLOY=' /opt/universe/.env</code>	no output (absent; grep exits 1), or <code>SEED_ON_DEPLOY=reference</code> — never 1 / reset	No fabricated treaties can be seeded

#	Check	Command	Expected	Proves
C6	The app loaded <i>that</i> file	<code>sudo -u universe -H pm2 logs universe -l -lines 300 --nostream 2>/dev/null grep 'startup configuration loaded' tail -n 1</code>	newest line (format 0 universe <date>: {json}) with "envFile":"/opt/universe/.env"	The running process read this env file, not another

7.2 Connectivity — the database answers with the app's own credentials

#	Check	Command	Expected	Proves
D1	Connect with the app's URL	<code>psql "\$DATABASE_URL" -c 'select 1'</code>	one row, 1	Host, port, role, password and database name are right
D2	Identify what you are connected to	<code>psql "\$DATABASE_URL" -tAc "select current_database(), current_user, inet_server_addr(), inet_server_port(), version()"</code>	universe universe 127.0.0.1 5432 PostgreSQL 16... (your values)	You are looking at the intended database, not a stray one
D3	Postgres is up and listening locally	<code>sudo systemctl status postgresql --no-pager head -3 and ss -tlnp grep 5432 (alternatives: pg_lsclusters; psql "\$DATABASE_URL" -tAc "show listen_addresses")</code>	active (running); 127.0.0.1:5432 only (pg_lsclusters → online; listen_addresses → localhost)	Database service healthy and not exposed
D4	Role owns the schema	<code>psql "\$DATABASE_URL" -tAc "select pg_get_userbyid(datdba) from pg_database where datname=current_database()"</code>	universe	Migrations can create/alter objects without superuser help

7.3 Runtime — the *running* application is connected to *that* database

#	Check	Command	Expected	Proves
R1	Boot log names the database	<code>sudo -u universe -H pm2 logs universe -l -lines 300 --nostream 2>/dev/null grep 'DB pool connecting' tail -n 1</code>	"url":"postgresql://universe:***@localhost:5432/universe" matching DATABASE_URL (password masked)	The process is using the same host/port/db you tested in D1–D2

#	Check	Command	Expected	Proves
R2	Boot sequence completed	same log, <code>grep -e 'database connection verified' -e 'migrations skipped' -e 'migrations complete' -e 'reference data ready' -e 'http server listening' tail -n 12</code>	For the most recent start, once per worker : database connection verified, migrations skipped, reference data ready (appears twice per worker), http server listening. migrations skipped is correct — the deploy applied them; migrations complete appears instead only if <code>RUN_MIGRATIONS_ON_BOOT=true</code>	App verified the DB, found the schema, seeded, and is serving
R3	Deep health via the app port	<code>curl -sS -D - http://127.0.0.1:4000/api/health/deep</code>	HTTP 200, "db": {"ok":true,"pingMs":<small>,"pool":{...,"max":<per-worker pool>,...}}, header X-Pool-Waiting: 0. Under <code>ecosystem.config.js</code> the per-worker pool is <code>floor(80 / workers)</code> — 20 on a 4-worker box, 40 with 2 workers; 50 only when <code>DB_POOL_MAX</code> is set or the app runs outside the ecosystem file (see P1)	Live round-trip from the app to the DB, pool not saturated
R4	Deep health through nginx/TLS	<code>curl -sS https://<APP_DOMAIN>/api/health/deep</code>	same body, "env": "production"	Users' path reaches the same connected app
R5	DB sees the app's connections	<code>psql "\$DATABASE_URL" -c "select username, client_addr, state, count(*) from pg_stat_activity where datname=current_database() group by 1,2,3"</code>	rows with username = universe (idle/active) from the app host	The connections exist in Postgres, from the expected role and host
R6	Read path: app data = DB data	<code>curl -sS http://127.0.0.1:4000/api/auth/users grep -o '"username":"[^"]*"' cut -d'"' -f4 sort vs psql "\$DATABASE_URL" -tAc "select username from public.uw_user where is_active order by 1"</code>	identical lists (and not the two-user demo fallback <code>cuo / underwriter</code>)	The login screen is served from this database

#	Check	Command	Expected	Proves
R7	Write path: a login lands in the audit log	Log in once in the browser, then <code>psql "\$DATABASE_URL" -c "select event_type, payload->'actor' ->'name' as username, actor as user_id, created_at from public.audit_log where event_type='LOGIN' order by created_at desc limit 3"</code>	top row: <code>username =</code> the account you just used (the <code>actor</code> column holds its user id), <code>created_at =</code> just now	The app writes to this database, and the clock/timezone are sane
R8	Smoke test	<code>BASE_URL=https://<APP_DOMAIN>/opt/universe/deploy/smoke-test.sh</code> (add <code>SMOKE_USER / SMOKE_PASSWORD</code> to test a real login)	PASSED	All of R3–R6 in one exit code

7.4 Schema alignment — the code and the database agree on the migrations

#	Check	Command	Expected	Proves
S1	Nothing pending	<code>cd /opt/universe && sudo -u universe -H npm run migrate:status --prefix server tail -n 3</code>	Pending (0):	Every migration file in this checkout has been applied
S2	Counts match	<code>psql "\$DATABASE_URL" -tAc "select count(*) from public._migrations" vs ls /opt/universe/server/src/db/migrations/*.sql wc -l</code>	equal (149 at this commit)	No file applied twice, none missing
S3	No orphans (DB ahead of code)	<code>psql "\$DATABASE_URL" -tAc "select filename from public._migrations" sort > /tmp/applied.txt; ls /opt/universe/server/src/db/migrations sort > /tmp/ondisk.txt; comm -23 /tmp/applied.txt /tmp/ondisk.txt</code>	empty	The database was never migrated by code this checkout does not contain. If a filename prints, the old repository shipped a migration the new one lacks — stop and reconcile before deploying further (docs/migration-audit.md)
S4	No stragglers (code ahead of DB)	<code>comm -13 /tmp/applied.txt /tmp/ondisk.txt</code>	empty	Same as S1, file-by-file
S5	Last migration is recent and expected	<code>psql "\$DATABASE_URL" -c "select filename, applied_at from public._migrations order by applied_at desc limit 5"</code>	newest = <code>157_quote_offer_status_checks.sql</code> at this commit, <code>applied_at =</code> your deploy time	The deploy's migration step ran against this database

#	Check	Command	Expected	Proves
S6	Core tables answer	<code>psql "\$DATABASE_URL" -c "select (select count(*) from public.contract) contracts, (select count(*) from public.quote) quotes, (select count(*) from public.claim) claims, (select count(*) from public.uw_user) users"</code>	your real business counts, unchanged from before the switch	The data survived; the app's tables exist

7.5 Reference-data alignment — the lookups the app expects are present

#	Check	Command	Expected	Proves
F1	Reference counts	<code>cd /opt/universe && sudo -u universe -H npm run seed:reference:check</code>	the §5.2 block: countries 75, currencies ≥32, brokers 11, reinsurers 25, cedants ≥40, treaty types 11, classes 15, roles 9; business rows = yours	The reference seed reached this database
F2	Saudi/GCC cedants visible	SQL in §5.2	Tawuniya, MEDGULF, ... listed	GCC extras present
F3	No fabricated treaties	<code>psql "\$DATABASE_URL" -tAc "select count(*) from public.contract where import_metadata->>'source'='seed:test-treaties'"</code>	0	The test portfolio was never loaded
F4	UI dropdowns	New treaty screen: Cedant, Currency, Reinsurer, Class of business, Treaty type	populated as in §5.2	The app reads the seeded lookups

7.6 Capacity and users

#	Check	Command	Expected	Proves
P1	Connection budget	<code>psql "\$DATABASE_URL" -tAc "show max_connections" ; PM2 workers from pm2 status ; per-worker pool from R3 pool.max ; grep -E '^DB_POOL_MAX=' /opt/universe/.env</code>	workers × pool.max ≤ max_connections – 20 (e.g. 4 × 20 = 80 ≤ 80). ecosystem.config.cjs derives 20 per worker on a 4-core box only while DB_POOL_MAX is not set in .env — a DB_POOL_MAX=50 line (as in older notes) is used verbatim per worker: 4 × 50 = 200 > 100. Remove it or set it to the per-worker value	The cluster cannot exhaust Postgres

#	Check	Command	Expected	Proves
P2	Accounts	<code>cd /opt/universe/server && sudo -u universe -H node scripts/manage-user.js list</code>	your real users active; seeded personas (<code>chief.underwriter</code> , <code>retro.manager</code> , <code>underwriter1-4</code>) rotated or deactivated	No <code>demo2026</code> login remains
P3	Backup after the switch	<code>ls -lt /var/backups/universe head -2 ; weekly scripts/verify-restore.sh exit 0</code>	a dump newer than the deploy. <code>verify-restore.sh</code> creates and drops a scratch database, so the role in its <code>DATABASE_URL</code> needs <code>CREATEDB</code> — grant it once with <code>sudo -u postgres psql -c 'ALTER ROLE universe CREATEDB'</code> (as <code>deploy/README.md</code> step 10 does), or run it with a superuser URL; otherwise it fails with <code>permission denied to create database</code>	Recovery point reflects the migrated schema

7.7 Sign-off summary

- §7.1 C1–C6: one env file, one `DATABASE_URL` , production mode, seeding posture correct, app loaded that file.
- §7.2 D1–D4: `psql` with the app's URL works and identifies the intended database; role owns it.
- §7.3 R1–R8: boot log, deep health (direct and via nginx), `pg_stat_activity` , user list = `uw_user` , login audited, smoke test `PASSED` .
- §7.4 S1–S6: `Pending (0)` , counts equal, no orphans, no stragglers, last migration as expected, business counts unchanged.
- §7.5 F1–F4: reference counts as in §5.2, GCC cedants visible, zero fabricated treaties, dropdowns populated.
- §7.6 P1–P3: connection budget, accounts rotated, fresh backup.

8. Troubleshooting the switch

Symptom	Cause	Fix
<code>remote: Repository not found / fatal: Authentication failed on fetch or ls-remote</code>	Old token (scoped to <code>modelling_tool</code>) still in <code>~universe/.git-credentials</code> , or the new token's resource owner/repository is wrong	Remove the <code>github.com</code> line from the credential file (§3.1), mint the token with resource owner Saudi_re and repository Saudi_re , Contents: Read; retry <code>git ls-remote</code> interactively as <code>universe</code>
<code>Permission denied (publickey)</code>	Deploy key not added to <code>Saudi_re</code> , or the SSH alias/IdentityFile is wrong	§3.2; a deploy key works for one repository only — generate a new key for <code>Saudi_re</code>

Symptom	Cause	Fix
remote: Invalid username or token. Password authentication is not supported	An account password was typed at the prompt	Paste the fine-grained token as the password (v2.0 §12.1)
deploy.sh stops at DATABASE_URL is not set (create /etc/universe/universe.env ...)	The script defaults to the kit's env file; your v2.0 server keeps it at /opt/universe/.env	Run it as <code>sudo env ENV_FILE=/opt/universe/.env /opt/universe/deploy/deploy.sh --ref main --no-restart</code>
deploy.sh stops at run as root (sudo) or as universe	Started as your admin user without sudo	Prefix with <code>sudo</code> (it drops to <code>universe</code> for the Git/npm/build/migrate/seed steps itself)
Failed to restart universe.service: Unit universe.service not found	deploy.sh was run without <code>--no-restart</code> on a PM2 server	Nothing is broken — migrations and seed already ran. Reload PM2: <code>sudo -u universe -H bash -c 'cd /opt/universe && pm2 reload ecosystem.config.cjs'</code>
<code>sudo -u universe -H pm2 ...</code> → <code>pm2: command not found</code>	PM2 is not installed globally for that user (only as the repository's devDependency)	Use <code>/opt/universe/node_modules/.bin/pm2</code> in place of <code>pm2</code> , or install it globally as v2.0 §3.1 did (<code>sudo npm install -g pm2</code>)
<code>scripts/verify-restore.sh</code> fails with <code>permission denied to create database</code>	The <code>universe</code> role cannot create the scratch database the restore drill uses	<code>sudo -u postgres psql -c 'ALTER ROLE universe CREATEDB'</code> once, or run the drill with a superuser <code>DATABASE_URL</code> (§7.6 P3)
deploy.sh errors while reading the env file (<code>command not found</code> , <code>unexpected token</code>)	A value in <code>.env</code> contains shell-special characters (<code>&</code> , spaces, <code>\$</code>) and the script sources the file	Quote that value in <code>.env</code> (<code>KEY='value'</code>), or export <code>DATABASE_URL</code> in the shell and rerun
vite: not found during the client build	<code>npm ci</code> ran with <code>NODE_ENV=production</code> exported and skipped devDependencies	Use <code>deploy.sh</code> (it passes <code>--include=dev</code>) or <code>npm ci --include=dev --prefix client</code>
<code>migrate:status</code> still shows pending files after the deploy	Migration step failed earlier (read its error), or it ran against a different <code>DATABASE_URL</code> than the app uses	Fix the cause, <code>npm run migrate:up --prefix server</code> , then §7.4 S1–S4
§7.4 S3 prints a filename (orphan migration)	The database was migrated by a version of the old repository that shipped a file this checkout lacks	Do not "fix" the <code>_migrations</code> table. Compare the old checkout's <code>server/src/db/migrations/</code> with the new one, port the missing file forward as a new additive migration, and follow <code>docs/migration-audit.md</code>
Seed fails: no unique or exclusion constraint matching the ON CONFLICT specification	Migration 150 not applied yet	Run <code>migrate:up</code> first; the seed must follow the migrations
Seed aborts: <code>seed:reference</code> changed business table	A business row was written by a user during the run	Re-run at a quiet moment; the seed itself never writes those tables

Symptom	Cause	Fix
seed-on-deploy: SEED_ON_DEPLOY is not set – skipping.	Expected unless you wanted the Ghana pack	Add <code>SEED_ON_DEPLOY=reference</code> to <code>.env</code> and re-run <code>npm run seed:deploy --prefix server</code>
Fabricated treaties appeared on the dashboards	Someone set <code>SEED_ON_DEPLOY=1</code> / reset or ran <code>seed:treaties</code>	<code>node server/scripts/seedTestTreaties.js --reset-only</code> removes only rows stamped <code>seed:test-treaties</code> ; remove the variable
Login screen lists only <code>cuo</code> and <code>underwriter</code>	The auth route fell back to demo users because the tables were not ready — migrations did not run, or the app points at an empty database	\$7.3 R1 + \$7.4 S1; fix <code>DATABASE_URL</code> or run <code>migrate:up</code> , reload
<code>/api/health/deep</code> returns 503 <code>db.ok false</code>	Postgres down, wrong <code>DATABASE_URL</code> , or pool exhausted	\$7.2 D1–D3; <code>X-Pool-Waiting > 0</code> sustained → raise <code>DB_POOL_MAX</code> within \$7.6 P1
Postgres logs / app errors <code>FATAL: sorry, too many clients already</code>	<code>workers × DB_POOL_MAX</code> exceeds <code>max_connections</code> (typically <code>DB_POOL_MAX=50</code> left in <code>.env</code> under a 4-worker cluster)	\$7.6 P1: remove <code>DB_POOL_MAX</code> from <code>.env</code> (the ecosystem file derives a safe per-worker value) or set it to ≤ 20 , then <code>pm2 reload ecosystem.config.cjs</code>
PM2 prints In-memory PM2 is out-of-date / version mismatch after the deploy	The repository's <code>npm ci</code> installed a local <code>pm2</code> that differs from the daemon's version	Keep using the <code>pm2</code> you started the app with; if you want them aligned run <code>pm2 update</code> (as the PM2 user) — it restarts the daemon, not the app
App starts with <code>AUTH_JWT_SECRET</code> is required / must not be a placeholder	Env file incomplete	v2.0 §5.1 — set a ≥ 32 -char secret, reload
Page looks unchanged after the deploy	Browser cache	Hard refresh (Ctrl/Cmd-Shift-R); confirm <code>git log -1</code> and PM2 uptime

Appendix A — Quick command reference (v2.0 layout, PM2)

Task	Command
Switch remote	<code>sudo -u universe -H git -C /opt/universe remote set-url origin https://github.com/iwadaya/Saudi_re.git</code>
Full deploy, keep PM2	<code>sudo env ENV_FILE=/opt/universe/.env /opt/universe/deploy/deploy.sh --ref main --no-restart</code>
Reload app (zero downtime)	<code>sudo -u universe -H bash -c 'cd /opt/universe && pm2 reload ecosystem.config.cjs && pm2 save'</code>
Smoke test	<code>BASE_URL=http://127.0.0.1:4000 /opt/universe/deploy/smoke-test.sh</code>
Migration status / apply	<code>npm run migrate:status --prefix server / npm run migrate:up --prefix server (from /opt/universe, as universe)</code>
Reference seed / check	<code>node server/src/db/seeds/run.js / npm run seed:reference:check</code>

Task	Command
Ghana pack (optional)	<code>SEED_ON_DEPLOY=reference in .env , then npm run seed:deploy --prefix server</code>
Remove fabricated treaties	<code>node server/scripts/seedTestTreaties.js --reset-only</code>
Users	<code>cd server && node scripts/manage-user.js list set-password <user> [--must-change] deactivate <user></code>
Health	<code>curl -s localhost:4000/api/health/deep</code>
Logs	<code>sudo -u universe -H pm2 logs universe --lines 100</code>
Backup / verify	<code>scripts/backup-db.sh / scripts/verify-restore.sh (with DATABASE_URL set)</code>
Rollback code	<code>... deploy/deploy.sh --ref <old commit> --no-restart --no-seed then reload</code>

Appendix B — Sign-off record

Item	Value
Date / time of switch	
Performed by	
Old commit (modelling_tool)	
New commit (Saudi_re main)	
Migrations applied (files)	
Reference counts (F1)	countries currencies brokers reinsurers cedants
Business counts before / after (S6)	contracts / quotes / claims / users /
Smoke test (R8)	PASSED ☐
Checklist §7.7 complete	☐
Backup taken after switch (P3)	file:
Token / deploy-key expiry	

Universe Reinsurance Platform — Confidential — For Internal IT Use Only — Repository Migration & Database Alignment Guide v1.0, September 2026. Always confirm against the latest code in `iwadaya/Saudi_re` before relying on this snapshot.